JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DoubleSummaryStatistics

java.lang.Object
    java.util.DoubleSummaryStatistics

All Implemented Interfaces:

DoubleConsumer

```
public class DoubleSummaryStatistics
extends Object
implements DoubleConsumer
```

A state object for collecting statistics such as count, min, max, sum, and average.

This class is designed to work with (though does not require) streams. For example, you can compute summary statistics on a stream of doubles with:

```
DoubleSummaryStatistics stats = doubleStream.collect(DoubleSummaryStatistics::ne
                                                     DoubleSummaryStatistics::ac
                                                     DoubleSummaryStatistics::co
```

`DoubleSummaryStatistics` can be used as a reduction target for a stream. For example:

```
DoubleSummaryStatistics stats = people.stream()
    .collect(Collectors.summarizingDouble(Person::getWeight));
```

This computes, in a single pass, the count of people, as well as the minimum, maximum, sum, and average of their weights.

Implementation Note:

This implementation is not thread safe. However, it is safe to use `Collectors.summarizingDouble()` on a parallel stream, because the parallel implementation of `Stream.collect()` provides the necessary partitioning, isolation, and merging of results for safe and efficient parallel execution.

This implementation does not check for overflow of the count.

Since:

1.8

Constructor Summary

Constructors

Constructor	Description
`DoubleSummaryStatistics()`	Constructs an empty instance with zero count, zero sum, `Double.POSITIVE_INFINITY` min, `Double.NEGATIVE_INFINITY` max and zero average.
`DoubleSummaryStatistics (long count, double min, double max, double sum)`	Constructs a non-empty instance with the specified `count`, `min`, `max`, and `sum`.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`void`	`accept(double value)`	Records another value into the summary information.
`void`	`combine (DoubleSummaryStatistics`	Combines the state of another `DoubleSummaryStatistics` into this one.
`final double`	`getAverage()`	Returns the arithmetic mean of values recorded, or zero if no values have been recorded.
`final long`	`getCount()`	Return the count of values recorded.
`final double`	`getMax()`	Returns the maximum recorded value, `Double.NaN` if any recorded value was NaN or `Double.NEGATIVE_INFINITY` if no values were recorded.
`final double`	`getMin()`	Returns the minimum recorded value,

		`Double.NaN` if any recorded value was NaN or `Double.POSITIVE_INFINITY` if no values were recorded.
`final double`	**`getSum`**`()`	Returns the sum of values recorded, or zero if no values have been recorded.
`String`	**`toString`**`()`	Returns a non-empty string representation of this object suitable for debugging.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods declared in interface DoubleConsumer

`andThen`

Constructor Details

DoubleSummaryStatistics

```
public DoubleSummaryStatistics()
```

Constructs an empty instance with zero count, zero sum, `Double.POSITIVE_INFINITY` min, `Double.NEGATIVE_INFINITY` max and zero average.

DoubleSummaryStatistics

```
public DoubleSummaryStatistics(long count,
                               double min,
                               double max,
                               double sum)
                        throws IllegalArgumentException
```

Constructs a non-empty instance with the specified `count`, `min`, `max`, and `sum`.

If `count` is zero then the remaining arguments are ignored and an empty instance is constructed.

If the arguments are inconsistent then an `IllegalArgumentException` is thrown. The necessary consistent argument conditions are:

- `count >= 0`
- `(min <= max && !isNaN(sum)) || (isNaN(min) && isNaN(max) && isNaN(sum))`

API Note:

The enforcement of argument correctness means that the retrieved set of recorded values obtained from a `DoubleSummaryStatistics` source instance may not be a legal set of arguments for this constructor due to arithmetic overflow of the source's recorded count of values. The consistent argument conditions are not sufficient to prevent the creation of an internally inconsistent instance. An example of such a state would be an instance with: `count` = 2, `min` = 1, `max` = 2, and `sum` = 0.

Parameters:

`count` - the count of values

`min` - the minimum value

`max` - the maximum value

`sum` - the sum of all values

Throws:

`IllegalArgumentException` - if the arguments are inconsistent

Since:

10

Method Details

accept

`public void accept(double value)`

Records another value into the summary information.

Specified by:

`accept` in interface `DoubleConsumer`

Parameters:

`value` - the input value

combine

```
public void combine(DoubleSummaryStatistics other)
```

Combines the state of another `DoubleSummaryStatistics` into this one.

Parameters:

`other` - another `DoubleSummaryStatistics`

Throws:

`NullPointerException` - if `other` is null

getCount

```
public final long getCount()
```

Return the count of values recorded.

Returns:

the count of values

getSum

```
public final double getSum()
```

Returns the sum of values recorded, or zero if no values have been recorded.

The value of a floating-point sum is a function both of the input values as well as the order of addition operations. The order of addition operations of this method is intentionally not defined to allow for implementation flexibility to improve the speed and accuracy of the computed result. In particular, this method may be implemented using compensated summation or other technique to reduce the error bound in the numerical sum compared to a simple summation of `double` values. Because of the unspecified order of operations and the possibility of using differing summation schemes, the output of this method may vary on the same input values.

Various conditions can result in a non-finite sum being computed. This can occur even if the all the recorded values being summed are finite. If any recorded value is non-finite, the sum will be non-finite:

- If any recorded value is a NaN, then the final sum will be NaN.
- If the recorded values contain one or more infinities, the sum will be infinite or NaN.
    - If the recorded values contain infinities of opposite sign, the sum will be NaN.
    - If the recorded values contain infinities of one sign and an intermediate sum overflows to an infinity of the opposite sign, the sum may be NaN.

It is possible for intermediate sums of finite values to overflow into opposite-signed infinities; if that occurs, the final sum will be NaN even if the recorded values are all finite. If all the recorded values are zero, the sign of zero is *not* guaranteed to be preserved in the final sum.

API Note:

Values sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the sum of values, or zero if none

getMin

```
public final double getMin()
```

Returns the minimum recorded value, `Double.NaN` if any recorded value was NaN or `Double.POSITIVE_INFINITY` if no values were recorded. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero.

Returns:

the minimum recorded value, `Double.NaN` if any recorded value was NaN or `Double.POSITIVE_INFINITY` if no values were recorded

getMax

```
public final double getMax()
```

Returns the maximum recorded value, `Double.NaN` if any recorded value was NaN or `Double.NEGATIVE_INFINITY` if no values were recorded. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero.

Returns:

the maximum recorded value, `Double.NaN` if any recorded value was NaN or `Double.NEGATIVE_INFINITY` if no values were recorded

getAverage

```
public final double getAverage()
```

Returns the arithmetic mean of values recorded, or zero if no values have been recorded.

The computed average can vary numerically and have the special case behavior as computing the sum; see `getSum()` for details.

API Note:

Values sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the arithmetic mean of values, or zero if none

toString

```
public String toString()
```

Returns a non-empty string representation of this object suitable for debugging. The exact presentation format is unspecified and may vary between implementations and versions.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the object

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.